

UT_01.3. Funciones y programación funcional

Índice

- 3.1 Declaración de funciones
- 3.2 Funciones de orden superior
- 3.3 Literales de función
- 3.4 Funciones flecha
- 3.5 Procesamiento funcional de matrices
- 3.6 Modo estricto
- 3.7 Comprobación de tipos de argumentos
- 3.8 Proporcionar más o menos argumentos
- 3.9 Argumentos predeterminados
- 3.10 Parámetros restantes y el operador de propagación
- 3.11 Hoisting
- 3.12 Lanzamiento de excepciones
- 3.13 Captura de excepciones
- 3.14 La cláusula finally

En este tema aprenderás a escribir funciones en JavaScript. JavaScript es un lenguaje de programación «funcional». Las funciones son valores «de primera clase», al igual que los números o las cadenas. Las funciones pueden consumir y producir otras funciones. Dominar el estilo de programación funcional es esencial para trabajar con JavaScript moderno.

Este tema también cubre las reglas de paso de parámetros y ámbito de JavaScript, así como los detalles de lanzar y capturar excepciones.

3.1 Declaración de funciones

En JavaScript, se declara una función proporcionando

1. El nombre de la función
2. Los nombres de los parámetros
3. El cuerpo de la función, que calcula y devuelve el resultado de la función

No se especifican los tipos de los parámetros ni del resultado de la función. A continuación se muestra un ejemplo:

```
function average (x, y) {  
  return (x + y) / 2 }  
}
```

La instrucción `return` envía el valor que devuelve la función.

Para llamar a esta función, simplemente pase los argumentos deseados:

```
let result = average(6, 7) // el resultado se establece en 6,5
```

¿Qué ocurre si se pasa algo que no sea un número? Pase lo que pase, pasa. Por ejemplo:

```
result = average ('6', '7') // el resultado se establece en 33,5
```

Cuando pasas cadenas, el signo + en el cuerpo de la función las concatena. La cadena resultante «67» se convierte en un número antes de la división por 2.

Esto puede parecer bastante informal para un programador de Java, C# o C++, acostumbrado a la comprobación de tipos en tiempo de compilación. De hecho, si se confunden los tipos de argumentos, solo se descubre cuando ocurre algo extraño en tiempo de ejecución. Por otro lado, se pueden escribir funciones que funcionen con argumentos de varios tipos, lo que puede resultar conveniente.

La instrucción `return` devuelve inmediatamente, abandonando el resto de la función. Considera este ejemplo: una función `indexOf` que calcula el índice de un valor en una matriz:

```
function indexOf(arr, valor) {  
  for (let i in arr) {  
    if (arr[i] === valor) return i  
  }  
  return -1  
}
```

Tan pronto como se encuentra una coincidencia, se devuelve el índice y la función finaliza.

Una función puede optar por no especificar un valor de retorno. Si el cuerpo de la función sale sin una instrucción de retorno, o si la palabra clave `return` no va seguida de una expresión, la función devuelve el valor `undefined`.



Consejo

Si una función a veces devuelve un resultado y otras veces no quieres devolver nada, sé explícito:

```
return undefined
```



Nota

Una instrucción `return` siempre debe tener al menos un token antes del final de la línea, para evitar la inserción automática de punto y coma. Por ejemplo, si una función devuelve un objeto, coloque al menos el corchete de apertura en la misma línea:

```
return {  
  average: (x + y) / 2,  
  max: Math.max(x, y),  
  . . .  
}
```

3.2 Funciones de orden superior

JavaScript es un lenguaje de programación funcional. Las funciones son valores que se pueden almacenar en variables, pasar como argumentos o devolver como resultados de funciones.

Por ejemplo, podemos almacenar la función promedio en una variable:

```
let f = average
```

A continuación, se puede llamar a la función:

```
let result = f(6, 7)
```

Cuando se ejecuta la expresión `f(6, 7)`, se comprueba que el contenido de `f` es una función. Esa función se invoca con los argumentos 6 y 7.

Más adelante podemos poner otra función en la variable f:

```
f = Math.max
```

Ahora, cuando calculas f(6, 7), la respuesta es 7, el resultado de llamar a Math.max con los argumentos proporcionados.

A continuación se muestra un ejemplo de cómo pasar una función como argumento. Si arr es una matriz, la llamada al método

```
arr.map(someFunction)
```

aplica la función proporcionada a todos los elementos y devuelve una matriz con los resultados recopilados (sin modificar la matriz original). Por ejemplo,

```
result = [0, 1, 2, 4].map(Math.sqrt)
establece el resultado en
[0, 1, 1.4142135623730951, 2]
```

El método map a veces se denomina *función de orden superior*: una función que consume otra función.

3.3 Literales de función

Continuemos con el ejemplo de la sección anterior. Supongamos que queremos multiplicar todos los elementos de la matriz por 10. Por supuesto, podemos escribir una función

```
function multiplyBy10(x) { return x * 10 }
```

Ahora podemos llamar:

```
result = [0, 1, 2, 4].map(multiplyBy10)
```

Pero parece un desperdicio declarar una nueva función solo para usarla una vez.

Es mejor utilizar una *función literal*. JavaScript tiene dos variantes sintácticas. Aquí está la primera:

```
result = [0, 1, 2, 4].map(function (x) { return 10 * x })
```

La sintaxis es sencilla. Se utiliza la misma sintaxis de función que antes, pero ahora se omite el nombre. La función literal es un valor que denota la función con la acción especificada. Ese valor se pasa al método map.

Por sí misma, la función literal no tiene nombre, al igual que la matriz literal [0, 1, 2, 4] no tiene nombre. Si quieres darle un nombre a la función, haz lo que siempre haces cuando quieres darle un nombre a algo: guárdala en una variable:

```
const average = function (x, y) { return (x + y) / 2 }
```



Consejo

Piensa en los literales de función anónimos como el caso «normal». Una función con nombre es una forma abreviada de declarar un literal de función y luego darle un nombre.

3.4 Funciones flecha

En la sección anterior, ha visto cómo declarar funciones literales con la palabra clave function. Existe una segunda forma más concisa que utiliza el operador =>, normalmente denominado «flecha»:

```
const average = (x, y) => (x + y) / 2
```

Las variables de los parámetros se proporcionan a la izquierda de la flecha y el valor de retorno a la derecha.

Si hay un solo parámetro, no es necesario encerrarlo entre paréntesis:

```
const multiplyBy10 = x => x * 10
```

Si la función no tiene parámetros, utilice un conjunto de paréntesis vacíos:

```
const dieToss = () => Math.trunc(Math.random() * 6) + 1
```

Ten en cuenta que `dieToss` es una función, no un número. Cada vez que llamas a `dieToss()`, obtienes un número entero aleatorio entre 1 y 6.

Si una función flecha es más compleja, coloque su cuerpo dentro de una instrucción de bloque. Utilice la palabra clave `return` para devolver un valor fuera del bloque:

```
const indexOf = (arr, value) => {  
  for (let i in arr) {  
    if (arr[i] === value) return i  
  }  
  return -1  
}
```



Consejo

El token

`=>`

debe estar en la misma línea que los parámetros:

```
const average = (x, y)=> // OK  
  (x + y) / 2  
const distance = (x, y) // Error  
=> Math.abs(x - y)
```

Si escribes una función flecha en más de una línea, es más claro utilizar llaves:

```
const average = (x, y) => {  
  return (x + y) / 2  
}
```



Precaución

Si una función flecha no hace nada más que devolver un objeto literal, debe encerrar el objeto entre paréntesis:

```
const stats = (x, y) => ({  
  average: (x + y) / 2,  
  distance: Math.abs(x - y)  
})
```

De lo contrario, las llaves se analizarían como un bloque.



Consejo

Las funciones flecha tienen un comportamiento más regular que las funciones declaradas con la palabra clave `function`. Muchos programadores de JavaScript prefieren utilizar la sintaxis flecha para funciones anónimas y anidadas. Algunos programadores utilizan la sintaxis flecha para todas las funciones,

mientras que otros prefieren declarar las funciones de nivel superior con `function`. Se trata simplemente de una cuestión de gustos.

3.5 Procesamiento funcional de matrices

En lugar de iterar sobre una matriz con un bucle `for of` o `for in`, puede utilizar el método `forEach`. Pase una función que procese los elementos y los valores de índice:

```
arr.forEach((element, index) => { console.log(`${index}: ${element}`) })
```

La función se llama para cada elemento de la matriz, en orden ascendente de índice.

Si solo le interesan los elementos, puede pasar una función con un parámetro:

```
arr.forEach(element => { console.log(`${element}`) })
```

El método `forEach` llamará a esta función con el elemento y el índice, pero en este ejemplo, el índice se ignora.

El método `forEach` no produce ningún resultado. En su lugar, la función que le pasas debe tener algún efecto secundario, como imprimir un valor o realizar una asignación. Es incluso mejor si puedes evitar por completo los efectos secundarios y utilizar métodos como `map` y `filter`, que transforman las matrices en la forma deseada.

Ya vimos el método `map` que transforma una matriz, aplicando una función a cada elemento. Aquí tienes un ejemplo práctico. Supongamos que quieres crear una lista HTML de elementos en una matriz. Primero puedes encerrar cada uno de los elementos en un elemento `li`:

```
const enclose = (tag, contents) => `<${tag}>${contents}</${tag}>`  
const listItems = items.map(i => enclose('li', i))
```

En realidad, es más seguro escapar primero los caracteres `&` y `<` de los elementos. Supongamos que tenemos una función `htmlEscape` para este fin. Entonces, primero podemos transformar los elementos para que sean seguros y, a continuación, encerrarlos:

```
const listItems = items  
  .map(htmlEscape)  
  .map(i => enclose('li', i))
```

Ahora el resultado es una matriz de elementos `li`. A continuación, concatenamos todas las cadenas con el método `Array.join` y encerramos la cadena resultante en un elemento `ul`:

```
const list = enclose('ul',  
  items  
    .map(htmlEscape)  
    .map(i => enclose('li', i))  
    .join(''))
```

Otro método útil de matriz es `filter`. Recibe una función *predicado*, es decir, una función que devuelve un valor booleano (o *Bool-ish*). El resultado es una matriz con todos los elementos que cumplen el predicado. Siguiendo con el ejemplo anterior, no queremos incluir cadenas vacías en la lista. Podemos eliminarlas así:

```
const list = enclose('ul',  
  items  
    .filter(i => i.trim() !== '')  
    .map(htmlEscape)  
    .map(i => enclose('li', i))
```

```
.join(''))
```

Esta canalización de procesamiento es un buen ejemplo de un estilo de programación de alto nivel «qué, no cómo». ¿Qué queremos? Eliminar las cadenas vacías, escapar el HTML, encerrar los elementos en elementos `li` y unirlos. ¿Cómo se hace esto? En última instancia, mediante una secuencia de bucles y ramificaciones, pero eso es un detalle de implementación.

3.6 Modo estricto

Como has visto, JavaScript tiene algunas características inusuales, algunas de las cuales han demostrado ser poco adecuadas para el desarrollo de software a gran escala. *El modo estricto* prohíbe algunas de estas características. Siempre debes utilizar el modo estricto.

Para habilitar el modo estricto, coloque la línea

```
'use strict'
```

como primera línea que no sea un comentario en tu archivo. (Se pueden utilizar comillas dobles en lugar de simples, al igual que un punto y coma).

Si quieres forzar el modo estricto en el REPL de Node.js, inícialo con

```
node --use-strict
```



Nota

En la consola de un navegador, debes anteponer «use strict» a cada línea que desees ejecutar en modo estricto, o «use strict» seguido de Mayús+Intro. Esto no es muy práctico.

Puedes aplicar el modo estricto a funciones individuales:

```
function strictInASeaOfSloppy() {  
  'use strict'  
  . . .  
}
```

No hay ninguna razón válida para utilizar el modo estricto por función con código moderno. Aplique el modo estricto a todo el archivo.

A modo de resumen, estas son las características clave del modo estricto:

- Asignar un valor a una variable no declarada previamente es un error y no crea una variable global. Debe utilizar `let`, `const` o `var` para todas las declaraciones de variables.
- No se puede asignar un nuevo valor a una propiedad global de solo lectura, como `NaN` o `undefined`. (Lamentablemente, aún se pueden declarar variables locales que las oculten).
- Las funciones solo se pueden declarar en el nivel superior de un script o función, no en un bloque anidado.
- El operador `delete` no se puede aplicar a «identificadores no calificados». Por ejemplo, `delete parseInt` es un error de sintaxis. Intentar eliminar una propiedad que no es «configurable» (como `delete 'Hello'.length`) provoca un error de tiempo de ejecución.
- No se pueden tener parámetros de función duplicados (`function average(x, x)`). Por supuesto, nunca se han querido, pero son legales en el modo «descuidado» (no estricto).

- No se pueden utilizar literales octales con el prefijo 0: 010 es un error de sintaxis, no un 10 octal (que es 8 en decimal). Si se desea un octal, utilice 0o10.
- La instrucción with está prohibida.



Nota

En modo estricto, leer el valor de una variable no declarada genera un ReferenceError. Si necesitas saber si una variable ha sido declarada (e inicializada), no puedes comprobar

```
possiblyUndefinedVariable !== undefined
```

En su lugar, utilice la condición

```
typeof possiblyUndefinedVariable !== 'undefined'
```

3.7 Comprobación de tipos de argumentos

En JavaScript, no se especifican los tipos de argumentos de las funciones. Por lo tanto, puede permitir que los llamantes proporcionen un argumento de un tipo u otro, y manejar ese argumento según su tipo real.

Como ejemplo un tanto artificial, la función average puede aceptar números o matrices.

```
const average = (x, y) => {
  let sum = 0
  let n = 0
  if (Array.isArray(x)) {
    for (const value of x) { sum += value; n++ }
  } else {
    sum = x; n = 1
  }
  if (Array.isArray(y)) {
    for (const value of y) { sum += value }
  } else {
    sum += y; n++
  }
  return n === 0 ? 0 : sum / n
}
```

Ahora puedes llamar a:

```
result = average(1, 2)
result = average([1, 2, 3], 4)
result = average(1, [2, 3, 4])
result = average([1, 2], [3, 4, 5])
```

La **tabla 3-1** muestra cómo comprobar si un argumento x se ajusta a un tipo determinado.

Tabla 3-1 Pruebas de tipo

Tipo	Prueba	Notas
Cadena	<pre>typeof x === 'string' x instanceof String</pre>	x podría construirse como new String(...)

Tipo	Prueba	Notas
Expresión regular	<code>x instanceof RegExp</code>	
Número	<code>typeof x === 'number' x instanceof Number</code>	x podría construirse como <code>new Number(.)</code>
Cualquier cosa que se pueda convertir en un número	<code>typeof +x === 'number'</code>	Obtener el valor numérico como <code>+x</code>
Array	<code>Array.isArray(x)</code>	
Función	<code>typeof x === 'function'</code>	



Nota

Algunos programadores escriben funciones que convierten cualquier valor de argumento en números, como

```
const average = (x, y) => {
  return (+x + +y) / 2
}
```

Entonces se puede llamar a

```
average('3', [4])
```

¿Es ese grado de flexibilidad útil, inofensivo o un presagio de problemas? No lo recomiendo.

3.8 Proporcionar más o menos argumentos

Supongamos que una función se declara con un número concreto de parámetros, por ejemplo:

```
const average = (x, y) => (x + y) / 2
```

Parece que hay que proporcionar dos argumentos al llamar a la función. Sin embargo, esa no es la forma de hacerlo en JavaScript. Se puede llamar a la función con más argumentos, que se ignorarán silenciosamente:

```
let result = average(3, 4, 5) // 3,5: se ignora el último argumento
```

Por el contrario, si se proporcionan menos argumentos, los que faltan se establecen como indefinidos. Por ejemplo, `average(3)` es `(3 + indefinido) / 2`, o `NaN`. Si desea que esa llamada tenga un resultado significativo, puede hacer lo siguiente:

```
const average = (x, y) => y === undefined ? x : (x + y) / 2
```

3.9 Argumentos predeterminados

En la sección anterior, ha visto cómo implementar una función que se llama con menos argumentos que parámetros. En lugar de comprobar manualmente los valores de los argumentos indefinidos, puede proporcionar argumentos predeterminados en la declaración de la función. Después del parámetro, coloque un `=` y una expresión para el valor predeterminado, es decir, el valor que se debe utilizar si no se ha pasado ningún argumento.

Aquí hay otra forma de hacer que la función promedio funcione con un argumento:

```
const average = (x, y = x) => (x + y) / 2
```

Si llama a `average(3)`, entonces `y` se establece en `x`, es decir, 3, y se calcula el valor de retorno correcto.

Puedes proporcionar varios valores predeterminados:

```
const average = (x = 0, y = x) => (x + y) / 2
```

Ahora `average()` devuelve cero.

Incluso puede proporcionar un valor predeterminado para el primer parámetro y no para los demás:

```
const average = (x = 0, y) => y === undefined ? x : (x + y) / 2
```

Si no se proporciona ningún argumento (o se proporciona uno explícitamente indefinido), el parámetro se establece en el valor predeterminado o, si no se proporciona ninguno, en indefinido:

```
average(3) // average(3, undefined)
average() // average(0, undefined)
average(undefined, 3) // average(0, 3)
```

3.10 Parámetros restantes y el operador de propagación

Como ha visto, puede llamar a una función JavaScript con cualquier número de argumentos. Para procesarlos todos, declare el último parámetro de la función como un parámetro «*restante*» anteponiéndole el token `...`:

```
const average = (first = 0, ...following) => {
  let sum = first
  for (const value of following) { sum += value }
  return sum / (1 + following.length)
}
```

Cuando se llama a la función, el parámetro siguiente es una matriz que contiene todos los argumentos que no se han utilizado para inicializar los parámetros anteriores. Por ejemplo, consideremos la llamada:

```
average(1, 7, 2, 9)
```

Entonces, el primero es 1 y el siguiente es la matriz `[7, 2, 9]`.

Muchas funciones y métodos aceptan argumentos variables. Por ejemplo, el método `Math.max` devuelve el mayor de sus argumentos, sin importar cuántos haya:

```
let result = Math.max(3, 1, 4, 1, 5, 9, 2, 6) // Establece el resultado en 9
```

¿Qué pasa si los valores ya están en una matriz?

```
let numbers = [1, 7, 2, 9]
result = Math.max(numbers) // Da como resultado NaN
```

Eso no funciona. El método `Math.max` recibe una matriz con un elemento: la matriz `[1, 7, 2, 9]`.

En su lugar, utilice el operador «*spread*», el token `...` colocado antes de un *argumento* de matriz:

```
resultado = Math.max(...números) // Da como resultado 9
```

El operador *spread* distribuye los elementos como si se hubieran proporcionado por separado en la llamada.



Nota

Aunque el operador *spread* y la declaración *rest* parecen iguales, sus acciones son exactamente opuestas entre sí.

En primer lugar, tenga en cuenta que el operador de propagación se utiliza con un argumento, y la sintaxis de resto se aplica a una declaración de variable.

```
Math.max(...numbers) // Operador de propagación: argumento en la llamada a la función
const max = (...values) => { /* body */ }
// Declaración «rest» de la variable de parámetro
```

El operador de propagación convierte una matriz (o, de hecho, cualquier iterable) en una secuencia de valores. La declaración de *resto* hace que una secuencia de valores se coloque en una matriz.

Tenga en cuenta que puede utilizar el operador de propagación incluso si la función que llama no tiene ningún parámetro restante. Por ejemplo, considere la función promedio de la sección anterior que tiene dos parámetros. Si llama a

```
result = average(...numbers)
```

todos los elementos de `numeros` se pasan como argumentos a la función. La función utiliza los dos primeros argumentos e ignora los demás.



Nota

También puede utilizar el operador de propagación en un inicializador de matriz:

```
let moreNumbers = [1, 2, 3, ...numbers] // Operador de propagación
```

No confunda esto con la declaración *rest* utilizada con la desestructuración. La declaración *rest* se aplica a una variable:

```
let [first, ...following] = numbers // Declaración de resto
```



Consejo

Dado que las cadenas son iterables, puede utilizar el operador de extensión con una cadena:

```
let greeting = 'Hello 🌐'
let characters = [...greeting]
```

La matriz `characters` contiene las cadenas «H», «e», «l», «l», «o», « » y «🌐».

La sintaxis para los argumentos predeterminados y los parámetros restantes es igualmente aplicable a la sintaxis de la función:

```
function average(first = 0, ...following) { . . . }
```

3.11 Hoisting

Abordamos otro tema complejo que se puede evitar fácilmente siguiendo tres reglas sencillas. Son las siguientes:

- No utilices var
- Utilice el modo estricto
- Declara las variables y funciones antes de usarlas

Si quieres entender qué sucede cuando no sigues estas reglas, sigue leyendo.

JavaScript tiene un mecanismo inusual para determinar el *ámbito* de una variable, es decir, la región de un programa en la que se puede acceder a la variable. Considera una variable local, declarada dentro de una función. En lenguajes de programación como Java, C# o C++, el ámbito se extiende desde el punto en el que se declara la variable hasta el final del bloque que la contiene. En JavaScript, una variable local declarada con `let` parece tener el mismo comportamiento:

```
función doStuff() { // Inicio del bloque
  . . . // Al intentar acceder a someVariable se produce un ReferenceError
  let someVariable // El ámbito comienza aquí
  . . . // Se puede acceder a someVariable, el valor es indefinido
  someVariable = 42
  . . . // Se puede acceder a someVariable, el valor es 42
} // Fin del bloque, el ámbito termina aquí
```

Sin embargo, no es tan sencillo. Se *puede* acceder a las variables locales en funciones cuyas declaraciones preceden a la declaración de la variable:

```
función doStuff() {
  función localWork() {
    console.log(someVariable) // Se puede acceder a la variable
    . . .
  }
  let someVariable = 42

  localWork() // Imprime 42
}
```

En JavaScript, cada declaración se *eleva* a la parte superior de su ámbito. Es decir, se sabe que la variable o función existe incluso antes de su declaración, y se reserva espacio para almacenar su valor.

Dentro de una función anidada, se puede hacer referencia a variables o funciones elevadas. Consideremos la función `localWork` del ejemplo anterior. La función conoce la ubicación de `someVariable` porque se eleva a la parte superior del cuerpo de `doStuff`, aunque esa variable se declare después de la función.

Por supuesto, puede ocurrir que se acceda a una variable antes de ejecutar la instrucción que la declara. Con las declaraciones `let` y `const`, acceder a una variable antes de que se declare genera un `ReferenceError`. La variable se encuentra en la «zona muerta temporal» hasta que se ejecuta su declaración.

Sin embargo, si una variable se declara con la arcaica palabra clave `var`, su valor simplemente queda indefinido hasta que se inicializa la variable.



Consejo

No utilices `var`. Declara variables cuyo ámbito es toda la función, no el bloque que la contiene. Eso es demasiado amplio:

```
función someFunction(arr) {
  // i, elemento ya en el ámbito pero indefinido
```

```
for (var i = 0; i < arr.length; i++) {
  var elemento = arr[i]
  . . .
}
// i, elemento aún en el ámbito
}
```

Dado que las funciones se elevan, puedes llamar a una función antes de que se declare. En particular, puedes declarar funciones mutuamente recursivas:

```
función isEven(n) { return n === 0 ? true : isOdd(n -1) }
función isOdd(n) { return n === 0 ? false : isEven(n -1) }
```



Nota

En modo estricto, las funciones con nombre solo se pueden declarar en el nivel superior de un script o función, no dentro de un bloque anidado. En modo no estricto, las funciones con nombre anidadas se elevan a la parte superior de su función envolvente.

Siempre que utilices el modo estricto y evites las declaraciones var, es poco probable que el comportamiento de elevación provoque errores de programación. Sin embargo, es recomendable estructurar el código de manera que las variables y funciones se declaren antes de utilizarlas.



Nota

Antiguamente, los programadores de JavaScript utilizaban «funciones invocadas inmediatamente» para limitar el ámbito de las declaraciones var y las funciones:

```
(function () {
  var someVariable = 42
  function someFunction(. . .) { . . . }
  . . .
})(); // La función se llama aquí; tenga en cuenta los ()
// someVariable y someFunction ya no están en el ámbito
```

Después de llamar a la función anónima, nunca se vuelve a utilizar. Su único propósito es encapsular las declaraciones.

Este dispositivo ya no es necesario. Simplemente utilice:

```
{
  let someVariable = 42
  const someFunction = (. . .) => { . . . }
  . . .
}
```

Las declaraciones se limitan al bloque.

3.12 Lanzamiento de excepciones

Si una función no puede calcular un resultado, puede lanzar una excepción. Dependiendo del tipo de fallo, esta puede ser una estrategia mejor que devolver un valor de error como NaN o indefinido.

Utilice una instrucción throw para lanzar una excepción:

```
throw valor
```

El valor de la excepción puede ser un valor de cualquier tipo, pero lo habitual es lanzar un objeto de error. La función Error produce dicho objeto con una cadena determinada que describe el motivo.

```
let reason = `Element ${elem} not found`  
throw Error(reason)
```

Cuando se ejecuta la instrucción `throw`, la función se termina inmediatamente. No se produce ningún valor de retorno, ni siquiera indefinido. La ejecución no continúa en la llamada a la función, sino en la cláusula `catch` o `finally` más cercana, como se describe en las secciones siguientes.



Consejo

El manejo de excepciones es un buen mecanismo para situaciones impredecibles que el llamante podría no ser capaz de manejar. No es tan adecuado para situaciones en las que se espera un fallo. Considere el análisis de la entrada del usuario. Es muy probable que algunos usuarios proporcionen entradas inadecuadas. En JavaScript, es fácil devolver un valor «mínimo» como `undefined`, `null` o `NaN` (siempre que, por supuesto, no puedan ser entradas válidas). O bien, puede devolver un objeto que describa el éxito o el fallo.

3.13 Captura de excepciones

Para capturar una excepción, utilice una instrucción `try`. En **el tema 1.2**, ha visto cómo capturar una excepción si no le interesa el valor de la excepción. Si desea examinar el valor de la excepción, añada una variable a la cláusula `catch`:

```
try {  
    // Realizar trabajo  
    . . .  
} catch (e) {  
    // Gestionar excepciones  
    . . .  
}
```

La variable en la cláusula `catch` (aquí, `e`) contiene el valor de la excepción. Como se ha visto en la sección anterior, un valor de excepción es convencionalmente un objeto de error. Dicho objeto tiene dos propiedades: nombre y mensaje. Por ejemplo, si se llama a

```
JSON.parse('{ age: 42 }')
```

, se lanza una excepción con el nombre «`SyntaxError`» y el mensaje «Token inesperado a en JSON en la posición 2». (La cadena de este ejemplo no es válida en JSON porque la clave `age` no está entre comillas dobles).

El nombre de un objeto producido con la función `Error` es «`Error`». La máquina virtual JavaScript lanza errores con los nombres «`SyntaxError`», «`TypeError`», «`RangeError`», «`ReferenceError`», «`URIError`» o «`InternalError`».

En el controlador, puede registrar esa información en un lugar adecuado. Sin embargo, en JavaScript no suele ser productivo analizar el objeto de error en detalle, como se haría en lenguajes como Java o C++.

Cuando se registra un objeto de error en la consola, los entornos de ejecución de JavaScript suelen mostrar el *seguimiento de la pila*, es decir, las llamadas a funciones y métodos entre los puntos de lanzamiento y captura. Lamentablemente, no existe una forma estándar de acceder al seguimiento de la pila para registrarlo en otro lugar.



Nota

En Java y C++, puede capturar excepciones por su tipo. A continuación, puede gestionar errores de determinados tipos en un nivel bajo y otros en un nivel más alto. Estas estrategias no son fáciles de implementar en JavaScript. Una cláusula `catch` captura *todas* las excepciones, y los objetos de excepción contienen información limitada. En JavaScript, los controladores de excepciones suelen llevar a cabo una recuperación o limpieza genérica, sin intentar analizar la causa del fallo.

Cuando se introduce la cláusula `catch`, se considera que la excepción está gestionada. El procesamiento se reanuda con normalidad, ejecutando las sentencias de la cláusula `catch`. La cláusula `catch` puede salir con una sentencia `return` o `break`, o puede completarse ejecutando su última sentencia. En ese caso, la ejecución pasa a la siguiente sentencia después de la cláusula `catch`.

Si registra excepciones en un nivel de su código, pero trata el fallo en un nivel superior, entonces querrá *volver a lanzar* la excepción después de registrarla:

```
try {
  // Realizar el trabajo
  . . .
} catch (e) {
  console.log(e)
  throw e // Reenviar a un controlador que se ocupe del fallo
}
```

3.14 La cláusula `finally`

Una instrucción `try` puede tener opcionalmente una cláusula `finally`. El código de la cláusula `finally` se ejecuta independientemente de si se ha producido una excepción.

Veamos primero el caso más sencillo: una instrucción `try` con una cláusula `finally` pero sin cláusula `catch`:

```
try {
  // Adquirir recursos
  . . .
  // Realizar el trabajo
  . . .
} finally {
  // Liberar recursos
  . . .
}
```

La cláusula `finally` se ejecuta en todos los casos siguientes:

- Si todas las sentencias de la cláusula `try` se completaron sin lanzar una excepción
- Si se ejecutó una instrucción `return` o `break` en la cláusula `try`
- Si se produjo una excepción en cualquiera de las sentencias de la cláusula `try`

También puede tener una instrucción `try` con cláusulas `catch` y `finally`:

```
try {
  . . .
} catch (e) {
  . . .
} finally {
  . . .
}
```

Ahora hay una ruta adicional. Si se produce una excepción en la cláusula `try`, se ejecuta la cláusula `catch`. Independientemente de cómo salga la cláusula `catch` (de forma normal o mediante un `return/break/throw`), la cláusula `finally` se ejecuta a continuación.

El propósito de la cláusula `finally` es tener una única ubicación para liberar los recursos (como manejadores de archivos o conexiones de bases de datos) que se adquirieron en la cláusula `try`, independientemente de si se produjo una excepción o no.



Precaución

Es legal, pero confuso, tener sentencias `return/break/throw` en la cláusula `finally`. Estas sentencias tienen prioridad sobre cualquier sentencia de las cláusulas `try` y `catch`. Por ejemplo:

```
try {  
    // Realizar trabajo  
    . . .  
    return true  
} finally {  
    . . .  
    return false  
}
```

Si el bloque `try` tiene éxito y se ejecuta `return true`, se ejecuta la cláusula `finally`. Su `return false` enmascara la instrucción `return` anterior.